Degree Project in Computer Engineering

First cycle, 15 credits

Threshold Signatures vs Multi-Signatures

Investigating the Practical Tradeoff

MATYAS KOZAR

Threshold Signatures vs Multi-Signatures

Investigating the Practical Tradeoff

MATYAS KOZAR

Degree Programme in Computer Engineering
Date: March 19, 2026

Supervisor: Xavier Défago
Examiner: Francisco J. Peña
The School of Engineering Sciences in Chemistry, Biotechnology and Health
Host organization: Institute of Science Tokyo
Swedish title: Tröskelsignaturer kontra multisignaturer
Swedish subtitle: En undersökning av den praktiska avvägningen

Abstract

Distributed systems are crucial tools used in our daily lives, either directly or indirectly. In order to function properly, distributed system need to be able to handle a wide range of arbitrary faults (so called Byzantine faults). Protocols have been developed, to ensure these system remain operational in presence of Byzantine faults. A key part of these protocols is ensuring that information sent in the system can be trusted. This is done via digital signatures. Two different digital signature schemes are introduced in this paper: Multi-signatures and Threshold signatures. This study gives the following contributions: gives a clear numerical answer to the performance differences between Multi-signatures and Threshold signatures. Shows how each scheme performs in a real life Byzantine Fault tolerant protocol. Gives a clear and concise comparative analysis from the experimental results, and the practical trade-offs between each scheme, highlighting the strength and weaknesses by running benchmarks and simulations.

Keywords

Byzantine Fault Tolerance, Consensus Protocols, HotStuff, OMNeT++ Simulation, Threshold Signatures, Multi-signatures, Distributed Systems

Sammanfattning

Distribuerade system är avgörande verktyg som används i våra dagliga liv, antingen direkt eller indirekt. För att de ska fungera korrekt, behöver distribuerade system kunna hantera ett brett utbud av arbiträra fel (så kallade Bysantinskt fel). Protokoll har utvecklats, för att säkerställa att dessa system fortsätter vara funktionsdugliga i närvaron av Bysantinska fel. En huvuddel i dessa protokoll är att säkerställa att information som skickats i systemet kan förlitats. Det här görs genom digitala signaturer. Två olika digital signatur system är introducerad i denna rapport: Multisignaturer och tröskelsignaturer. Denna studie ger följande bidrag: ett tydlig numeriskt svar till skillnaderna i prestandan mellan multisignaturer och tröskelsignaturer. Visar hur varje system presterar i ett verkligt Bysantinskt feltolerans protokoll. Ger en tydlig och koncis komparativ analys från resultatet från experimenten, och praktiska avdelningar mellan varje system, betonar styrkor och svagheter genom att göra prestandamätningar och simulationer.

Nyckelord

Bysantinsk feltolerans, Konsensusprotokoll, HotStuff, OMNeT++-simulering, Tröskelsignaturer, Multi-signaturer, Distribuerade system

Acknowledgments

I would like to express my sincere gratitude to the D2S/Coord laboratory at Institute of Science Tokyo for providing me the opportunity and resources to conduct research needed for this thesis. Thanks to Professor Xavier Défago and Specially Appointed Associate Professor François Bonnet for their invaluable supervision, guidance and aid throughout my stay in Japan.

I am also thankful to Francisco J. Peña and Royal Institute of Technology (KTH) for the examination of this thesis, along with Anders Cajander for providing academic advice on writing this paper.

In addition, I wish to acknowledge the assistance from Takahiro Nakagawa, their help with providing the necessary code for running the simulations have been greatly appreciated.

Finally, I would like to give special thanks to my beloved family and friends, for continuously supporting me and giving me the encouragement needed to do this degree in the first place.

Stockholm, March 2026
Matyas Kozar

Contents

List of Figures

List of Tables

Acronyms and Abbreviations

BFT – **Byzantine Fault Tolerant**: The ability for a distributed system to remain functionally operational even when some of its nodes suffer arbitrary faults 1, 8, 14, 18, 20

DKG – **Distributed Key Generation**: A process where multiple parties contribute to a computation of a shared public/secret key pair 8, 9, 10, 23

EdDSA – **Edwards-curve Digital Signature Algorithm**: A digital signature scheme that uses a variant of a Schnorr signature based on twisted Edwards curves 18, 19

FROST – **Flexible Round-Optimized Schnorr Threshold Signatures**: A threshold signature scheme built upon the Schnorr signature scheme 9, 11, 12, 13, 18, 19, 20, 23, 24, 27

GST – **Global Stabilisation Time**: A specific point in time after all messages in a distributed system are guaranteed to be delivered within a bounded time frame xi, 15

KTH – **Royal Institute of Technology**: A university in Stockholm v

MPC – **Multiparty Computation**: A technique that allows multiple parties to jointly perform a single computation 8, 14

QC – **Quorum Certificate**: A collection of signatures from a threshold (quorum) required of signers, used in the context of blockchain and distributed systems vii, 20, 21, 28

ROAST – **RObust Asynchronous Schnorr Threshold Signatures**: A protocol that enhances existing Schnorr threshold signature schemes like FROST 12, 13, 18, 19, 20, 25, 28, 29

RSA – **Rivest–Shamir–Adleman**: A public-key encryption algorithm, widely used in digital signatures 7, 9

TSS – **Threshold Signature Scheme**: A method that allows a group of parties to jointly sign a message without requiring a centralised private key 1, 2, 8, 24, 28, 29

Chapter 1
Introduction

In an ever more connected world, seamless communication between people across the globe has become not just a convenience but a crucial aspect of how we function as a society. This reliance of a universally connected population relies heavily on the invisible backbone of distributed systems and network infrastructure. These systems ensure that information flows reliably, securely and seemingly instantaneously, no matter the distance. For this seamless communication to be possible, distributed systems must function at maximum level of reliability, ensuring that services remain available and performance remains constant. This however can only be possible in a perfect world; distributed systems are not perfect. In reality, faults occur, servers can fail, computers can malfunction, power can be cut at any moment, and malicious attacks can disrupt networks or slow connections. When the causes and effects of the faults are known, countermeasures can be implemented, either eliminating the chance of them occurring or minimising the impact on the system. But what about faults whose effects cannot be predicted? How can you ensure a distributed system functions correctly in the presence of unpredictable disruptions?

1.1. Problem Statement

Distributed systems resilient to these unpredictable disruptions are called Byzantine Fault Tolerant (BFT) systems. BFT systems are designed so that even in the scenario when some parts of the system are suffering from arbitrary faults (i.e., Byzantine faults), the system can still come to an agreement among multiple nodes on a single decision, a so called consensus. In order to agree on a consensus, usually a valid digital signature/certificate is needed, which is a cryptographic proof ensuring authenticity, integrity, and non-repudiation. The most common signature scheme for BFT systems are multi-signatures, however in the past decade, a newer signature scheme has gained attention: Threshold Signature Scheme (TSS.) Still, most BFT protocols opt out using TSS, since most of them were made initially with

multi-signatures as the foundation, but also because the advantages of using a TSS over multi-signatures have not been widely tested and documented.

1.2. Purpose

This report aims to produce a performance analysis between each signature scheme using defined comparison criteria, benchmarks and execution environment, in order to investigate the practicality and determine the best case scenario of each signature scheme, using a comparative analysis of the results.

1.3. Goals

- Comparative analysis (qualitative) with multi-signatures and threshold signatures
- Comparative analysis (quantitative: experimental)
- Implementation in HotStuff
- Comparison in terms of the impact on HotStuff
- Find the point in the growth of a system when multi-signatures are no longer realistic and threshold signatures can be considered instead

1.4. Ethical Approach

The author of the report hereby certifies that the report was produced without the aid of generative AI for purposes other than language checking.

1.5. Structure of the Thesis

This study is organised into the following chapters:

1. **Introduction:** This chapter.
2. **Background and Theory:** Briefly explains the required prerequisites, as well as introduces the signature schemes and protocols used in the experiments.
3. **Methodology:** Explains how the research was carried out, what hardware and software was used, what experiments were conducted and how they could be replicated.
4. **Results:** Showcases the results gathered from the methods used.
5. **Discussion:** Discusses the results from the experiments, and answers the questions from Chapter 1.

6. **Conclusions and Future Work:** Summarises the findings, highlights improvements and suggests future work.

Chapter 2
Background and Theory

Communication between multiple nodes in a distributed system, especially if they are geographically separated, can be volatile due to network unreliability, node failure and nodes acting independently or maliciously. Reliable distributed systems must thus be resilient to these types of faults, so that they can function even when one or more nodes might not be working properly.

2.1. Asymmetric Encryption

Asymmetric encryption, also known as public-key encryption, are cryptographic tools used to secure information [1]. Compared to symmetric cryptography, that uses a single shared key to both encrypt and decrypt information, asymmetric cryptography uses a public/secret key pair. The public key is used to encrypt the information, while the secret key (also known as the private key) is used to decrypt the information. Asymmetric cryptography needs to provide the following three functions:

- *generate():* A function that will generate the public/secret key pair. The secret key needs to be stored securely, where only the receiver can access it, while the public key needs to be stored somewhere where everyone that needs to encrypt the information can access it.
- *encrypt(plaintext, public key):* A function to encrypt plaintext into ciphertext using a public key.
- *decrypt(ciphertext, secret key):* A function that uses the secret key to decrypt the ciphertext back into plaintext.

The properties of these functions should be that if a piece of message in plaintext is encrypted, the resulting ciphertext, once decrypted, should yield the original message in plaintext.

2.2. Digital Signatures

Signatures are a crucial form of proof used to prove that a document is:

Authentic: The signer willingly signed the document and it has not been altered by anyone else.

Unforgeable: Proves that the signer themselves signed it and nobody else could copy their signature to impersonate them.

Non-repudiable: The signer cannot deny that they signed the document.

However, ensuring that physical signatures actually comply with the aforementioned points is nigh impossible; unlike their digital counterparts [1].

Just like actual signatures, digital signatures are used to prove the three[1] aspects of the message hold true. A digital signature should provide the following three functions:

- *generate():* Similar to the function in asymmetric encryption, but in contrast, the key functionality in digital signatures is flipped; the secret key is used to sign while the public key is used to verify the signature.
- *sign(m, SK):* A function to sign the message m with the secret key SK, producing the signature σ
- *verify(m, σ, PK):* A function to verify the authenticity of a signature σ on a message m, using the public key PK

Digital signatures sign a message m by using a mathematical algorithm and a secret key SK. To prove the integrity of the message, meaning it has not been altered after being signed, digital signatures use a compressed form of the message called a digest. It is a fixed-length value computed by applying a secure digest function (also known as a secure hash function). A secure hash function must satisfy the following two properties:

Collision resistance: Given the hash function H, it should be difficult[2] to find two messages m_1 and m_2 so that $H(m_1) = H(m_2)$.

Pre-image resistance: Given the hash h, it should be difficult to find a message m such that $h = H(m)$

[1] Assuming that the secret key has not been leaked, otherwise the sender could deny their signature, insisting that due to their secret key being leaked, it cannot prove that they themselves signed the message

[2] *Difficult* is defined as too computationally complex to solve in a satisfied timeframe, since with a finite amount of time, every cryptographic problem can eventually be solved

A signature σ can thus be generated by taking the digest d and encrypting it with SK and an algorithm, for example the Rivest–Shamir–Adleman (RSA) asymmetric cryptographic algorithm. When receiving m and σ, the receiver can verify σ by using the corresponding public key PK to decrypt σ, revealing the original d. Afterwards the receiver can run its own secure digest function on m to ensure it has not been tampered with during transit. By comparing the result of the digest with the one received in the message and see if they match, the receiver can tell if it is valid. If the two digests match, then it proves m has not been altered.

2.3. Faults in Distributed Systems

When discussing faults in distributed systems, there are three main ones to be concerned about:

Crash Fault: When a process completely stops functioning.

Crash faults are some of the most common faults and can be caused both by the software, like a program that gets stuck and thus stops receiving and giving instructions, or by the hardware, such as a server losing power.

Omission Fault: When a process fails to send or receive information.

In this context, a system/server might still be running, however, due to some unknown fault, for example network congestion, it is unable to receive or send information. This fault can be partial, meaning it is still operational but can occasionally miss an exchange.

Byzantine Fault: A fault in a system that is arbitrary/unpredictable.

Lastly, we have the core fault in this report, a Byzantine fault is a fault in a distributed system where a component behaves maliciously or unpredictably, such as crashes, sending incorrect information or purposely disrupting the system [2]. The name was coined by Leslie Lamport, from the “Byzantine Generals Problem”. A Byzantine fault is a broader category of faults that also includes the aforementioned two faults. A main concept in Byzantine fault, and the one that was used by Lamport in their paper, is equivocation: the act of sending different information to different receivers.

2.4. Byzantine Fault Tolerant Consensus Protocols

Many protocols have been developed to solve or mitigate the Byzantine Fault, these protocols are called BFT Consensus Protocols. While many differ in their solutions to this problem, they all share three main attributes: correct nodes agree on the same value, ensuring consistency. The value is actually proposed by a correct node, ensuring validity and all correct nodes will eventually come to a consensus, ensuring liveness.

A common theme in BFT Consensus Protocols is that they usually require a specific threshold t amount of nodes to come to a consensus (agreement) of the total system size n. This threshold is set as $\frac{2}{3}n$ (which is just a rewrite of $2f + 1$, where f stands for the number of faulty nodes), in order to ensure safety and liveness [3]. For each property, we have:

Safety: Two conflicting decisions cannot be both finalised due to any two quorums (minimum amount of nodes needed to sign/agree) of size $2f + 1$ must overlap in at least one non-faulty node that will not sign twice.

Liveness: Progress can always be made because quorums can be formed from $2f + 1$ nodes even if we have up to f nodes that are faulty.

2.4.1. Threshold Signatures

Threshold signatures are a Multiparty Computation (MPC) method [4], meaning it securely divides the general functionality among a set of nodes in order for all nodes to require to participate in the signing; some of the nodes are assumed to suffer from a Byzantine fault. It often refereed to as a t-out-of-n or (t, n)-threshold scheme. Thus the threshold part of the name comes from the fact that this signature scheme only requires a specific threshold of signers, instead of requiring every signer in the system to participate.

A TSS is made of n possible signers (nodes). First, a key needs to be generated. This can be done with a central trusted dealer (also known as a broker or coordinator). This node is a single entity that is trusted to perform the duty of generating the key and sending each node their respective secret share key and the joint public key. There is also an option to generate and distribute the keys, without a dealer, called Distributed Key Generation (DKG.) An example is Shamir's Secret Sharing algorithm [4].

When the key is generated, each node is given a secret key share, corresponding to a share of a single joint secret key. The set of signers all know a single joint public key. When the signing process is ongoing, a group of minimum t signers participate in the signing. The result is a joint signature of the message.

It offers two key properties: redundancy and corruption resilience. Redundancy in the way that it allows a threshold t number of nodes out of n total of nodes to issue a signature under a singular joint public key. Corruption resilience; if a maximum of $t - 1$ nodes are corrupted, then the signature remains unforgeable.

2.4.1.1. FROST & ROAST

One implementation of threshold signatures is the Flexible Round-Optimized Schnorr Threshold Signatures (FROST) scheme. FROST is a signature scheme that uses Schnorr signatures; a signature scheme utilising the Schnorr algorithm. This signature scheme is often used, due to providing good security and efficiency that results in significantly smaller signatures (approximately 212 bits, less than half the size of RSA signatures) and a faster operation time [5].

First, FROST needs a way to distribute the secret/public key pair to everyone, this is done with a DKG protocol:

FROST DKG Round 1

1. Every node n_i samples t random secret values, which act as coefficients of a secret-sharing polynomial of degree $t - 1$.
2. Every n_i computes a proof of knowledge.[3]
3. Every n_i computes a public commitment $\vec{C}_i$ by exponentiating a fixed base with each coefficient.
4. Every n_i sends to every other n_j their $\vec{C}_i$ and their proof of knowledge.
5. Upon receiving a $\vec{C}_i$ and n_i's proof, every n_j verifies the proof, aborting the protocol if it fails.
6. Once all commitments and proofs are verified, every n_i will discard any data used for that proof and only keeping the public commitments, which are used for the next round.

Figure 2.1: FROST's DKG protocol (Round 1).

FROST DKG Round 2

1. Every n_i sends to every n_j their secret share value for n_j, afterwards n_i will delete the sent share from its memory.
2. Every n_j will verify that the sent secret share matches the commitments published by n_i in the first round. If it fails, n_j will abort the protocol.
3. Every n_j will aggregate their share along with every other n_i's share to get their final secret share of the group's master key.
4. With their final secret share, each n_i computes a public verification share, which can then be multiplied together to form a public key, which is used to verify signatures.

Figure 2.2: FROST's DKG protocol (Round 2).

Once the DKG protocol is finished, the system will now be able to sign messages. FROST has both an option for a two round protocol with two messages sent in total or a single round signing protocol utilising the following preprocessing phase [6]:

[3] A proof computed by generating a random mask, computing a public commitment from that mask, hashing it together with their identity and committed value, which results in a *challenge* and lastly computing a response which ties that commitment with their secret.

FROST Preprocessing Round

Let j be a number for a nonce/commitment pair, and π be the number of pairs generated.

1. Construct an empty list L_i.
2. For each $1 \leq j \leq \pi$, do
   - Randomly choose two secret nonces
   - From each secret nonce, compute a public commitment
   - Save this pair to L_i, and save the corresponding secret locally
3. Publish L_i to a location where every other node n_i can fetch it.

Figure 2.3: FROST's preprocessing protocol.

Once the preprocessing round is finished, FROST can now initiate the signing round of a message m, using the commitments available in L_i:

FROST Sign

1. The coordinator S_C takes the next available commitment from each signer S_i from L_i and constructs a list B.
2. S_C sends to each S_i the message m to be signed along with B.
3. Upon receiving m and B, every S_i validates m and for every commitment in B, verifies they lie in the correct group.
4. Every S_i computes a binding value[4] for each S_j. Using the binding values and published commitments, each S_i aggregates all individual commitments, weighted by their binding values, into a group commitment (nonce point). This is then used to compute the challenge.
5. Every S_i, with their secret share, secret values, binding value and the challenge, computes their signature share.
6. Every S_i deletes the now-used commitment and the nonce point. Afterwards they send their signature share to S_C.
7. The S_C does the following upon receiving each signature share from S_i:
   - Derives the binding values and group commitment
   - Computes the challenge
   - Combines valid shares
   - Publishes the final signature σ along with m

Figure 2.4: FROST's single-round signing protocol.

According to the paper, FROST achieves an improved efficiency in scenarios where no signer misbehaves but also in cases where they do, it can identify these malicious signers and exclude them, rerunning the protocol afterwards. Since the pre-processing round is performed separately, it allows the protocol to be used asynchronously.

RObust Asynchronous Schnorr Threshold Signatures (ROAST) is an enhancement over FROST, specifically designed to act as a wrapper for FROST (or other Schnorr threshold signature schemes), simplifying the implementation of ROAST in systems already using FROST [7]. Although FROST is asynchronous in message flow, ROAST further improves on this by allowing multiple concurrent signing sessions. In FROST, each nonce can be used only

[4] A hash of the index of the signer, the message m and the list B

once, and each signer must coordinate on a specific session. In ROAST, multiple parallel signature requests can coexist without nonce reuse or requiring strict session tracking. Another key attribute and one that is very desirable in production environments is ROAST's ability to use retry mechanism in the presence of Byzantine faults. Despite the fact that FROST is able to detect malicious signers; it has no built-in automatic mechanisms to deal with them. The protocol must be re-run from start, with a new preprocessed nonce having to be generated since they are single-use. This adds unnecessary complexity and latency. In comparison, in ROAST, the nonces are pregenerated in advance, before any signing session begins, then committed to the coordinator. When a signing session is initiated, the coordinator simply chooses the threshold t amount of nonces that were already sent, and begins the session with the nodes that correspond to the sent nonces. Once a signer has sent their partial signature, they will piggyback the next nonce to the coordinator, to be used in future sessions. Once they do so, they will officially be done with the current signing session, even if the session is still waiting for other signers to respond, and thus be able to participate in a new signing session. In the case of a Byzantine fault, such as a delayed message, the signing session will simply wait for the delayed signer, until they respond or a timeout stops the session. Since the other signers are already done, they do not need to wait for the delayed signer to finish. This design gives the property of a signer of only being active in one session at a time, thus a signer can never be part of two or more sessions at once.

2.4.2. Multi-Signatures

Multi-signatures is a term used with schemes that combine a set of signatures, from a group of signers, into a single set of signatures (usually referred to as a multi-signature certificate) on a joint message [8]. Each signer has a secret/public key pair, and when signing a message, each signer produces their own signature σ. These signatures are then collected to produce a certificate. The final signature will be a certificate that includes every signature from every signer that took part in the signing process. Thus the resulting multi-signature is simply the aggregation of these n individual signatures, $\{\sigma_1, \sigma_2, \sigma_3, ..., \sigma_n\}$. While straightforward, this method results in a signature certificate whose size and verification time scale linearly with the number of participants.

2.5. HotStuff

HotStuff is a leader-based BFT protocol, commonly used in blockchain protocols [9]. It uses a chained three-phase commit architecture: a prepare phase, a pre-commit phase and a commit phase, chained by cryptographic hashes for efficiency. However it can also be simplified into just two phases, as shown by the HotStuff 2 paper [3]. In HotStuff, the leader proposes the next block; which is a data structure containing a set of transactions and metadata linking it to previous blocks [10]. The protocol follows a property called responsiveness; meaning it is being driven at the pace of actual network delay instead of maximum network delay [9]. This along with its linear complexity makes it stand out from other similar BFT protocols, as shown in Table 2.1.

Most other BFT protocols follow a two-phase paradigm, where a leader can drive a consensus decision in two rounds of message exchanges. In the first round, the leader proposes a value and collects a quorum certificate — a set of threshold signatures from a MPC system that proves consensus from a majority. The second round involves voting to confirm or finalise the proposal, with the goal of authenticating and committing the message. The problem with this approach is view-change; the algorithm responsible for replacing a leader. In two-phase paradigms, this algorithm adds an increased complexity, slowing down most protocols, especially with large system sizes. In HotStuff, a new leader simply has to pick the highest quorum certificate it knows about and the additional phase introduced allows nodes to change their vote if they desire so, without needing proof from the leader. This decreases the complexity and also makes leader replacement easier, at the cost of increased latency. Furthermore, the cost for a new leader to drive the protocol is equal to that of the current leader. This means that frequent leader replacement is fast, which is a desirable trait in blockchain contexts.

Table 2.1: Asymptotic complexity of selected protocols after GST [8]

Protocol	***Correct leader***	**Authenticator complexity** ***Leader failure (view-change)***	***f leader failures***	**Responsiveness**
DLS	$O(n^4)$	$O(n^4)$	$O(n^4)$	
PBFT	$O(n^2)$	$O(n^3)$	$O(fn^3)$	✓
SBFT	$O(n)$	$O(n^2)$	$O(fn^2)$	✓
Tendermint/ Casper	$O(n^2)$	$O(n^2)$	$O(fn^2)$	
Hotstuff	$\boldsymbol{O(n)}$	$\boldsymbol{O(n)}$	$\boldsymbol{O(fn)}$	✔

Chapter 3
Methodology

This chapter includes the methodologies used to compare the performance of multi-signature and threshold signature schemes. The primary objective is to establish a clear, reproducible and fair experimental framework. The research outline, selection of protocols, the experimental setup and the precise metrics for evaluation are outlined.

3.1. Research Design

This study employs a quantitative, comparative research design to systematically evaluate the performance of a multi-signature scheme and a threshold signature scheme. This approach was chosen due to it being the most direct and effective method that provides objective, data-driven answers to the goals in Chapter 1.3.

The study is quantitative in that it employs methods of collecting and analysing numerical data. The primary metrics, computation time and final signature size, provide concrete performance results that can be compared against each other.

The approach is comparative, due to the establishment of a controlled environment to conduct a side-by-side analysis of the two distinct signature schemes. By restricting both schemes to the same hardware, software and procedural conditions, the experiment isolates the performance differences to only the actual protocols' intrinsic designs.

Lastly, the design is empirical, meaning that all conclusions are derived from the results of these controlled experiments. This empirical foundation secures that the results are grounded in real-world performance characteristics, which offers practical insights into the trade-offs between the two signature schemes.

3.2. Protocol Selection and Implementation

3.2.1. The Multi-Signature Scheme

A naive multi-signature scheme was used, using the Ed25519 signature scheme, a variant of Edwards-curve Digital Signature Algorithm (EdDSA) . Although more complex variants of multi-signatures exist (e.g., aggregated multi-signatures), the reasoning for choosing this one is due to one of the main advantages of multi-signature schemes: simplicity. This choice represents a simple, widely understood baseline against which more complex schemes (i.e., FROST and ROAST) can be compared to.

3.2.2. The Threshold Signature Scheme

FROST was chosen due to its status as a modern, leading, efficient, and well-documented threshold signature scheme with available, high-quality open-source implementations, including a Rust rewrite of the original codebase in C++. This FROST implementation uses the same elliptic curve (Curve25519) but uses a true Schnorr signature, instead of the variant that the multi-signature scheme uses.

ROAST was chosen due being built upon FROST, therefore the same reasons mentioned above apply here too. The crucial difference is that ROAST is robust, offering retry-mechanisms in the presence of Byzantine faults, which is required for BFT protocols to function correctly.

3.2.3. HotStuff

HotStuff was chosen due to similar reasons as FROST and ROAST; being a modern, leading, efficient and well-documented BFT consensus protocol. A major advantage, as mentioned in Chapter 2, is the linear asymptotic complexity it offers compared to other BFT protocols. HotStuff improves on earlier protocols, offering a more simple, yet faster and secure BFT consensus protocol.

3.3. Experimental Environment

3.3.1. Hardware Specification

All benchmarks were conducted under the same hardware and software conditions:

- Device: Macbook Air M4 2025

- Processor: Apple M4 chip, 10-core CPU with 4 performance cores and 6 efficiency cores
- Graphics Card: Apple-designed integrated graphics 8-core GPU
- Memory: 16GB unified memory
- Storage: 256GB SSD

3.3.2. Software Specification

The following software was used:

- Device Operating System: macOS Sequoia v15.5
- Implementation Language: Rust v1.87.0 and C++ compiler supporting C++17.
- Benchmark Library: Criterion v0.3.0
- Simulation Library: OMNeT++ v6.2.0

The following cryptographic libraries were used:

- EdDSA: ed25519-dalek v2.0.0
- FROST: frost-ed25519 v2.1.0
- ROAST: a Rust rewrite of the ROAST library [11]
- Multi-signature: a modified Multi-Signature Crate

3.4. Evaluation Metrics and Measurement

To conduct a comprehensive and objective comparison, a set of quantitative and qualitative metrics was established. These metrics were chosen to capture the performance characteristics of each protocol as well as the practical considerations, from computational overhead to resource requirements.

3.4.1. Metric 1: Computation Time

This metric was directly chosen as the primary metric due to it reflecting the performance overhead imposed by the cryptographic operations on a system and is an important factor for throughput and latency. All measurements were recorded in nanoseconds (default for Criterion). The measurement is broken down into three distinct phases for each protocol, with an extra phase specifically for FROST.

3.4.1.1. Initialisation Time

The first phase is the initialisation. This measurement includes the total setup cost for a new signing group. The measurement begins at the invocation of the key generation function and concludes upon all n participants are

instantiated with their respective secret key/key share and the corresponding public key.

3.4.1.2. Signing Time

The second phase is the time it takes for a singular signer to sign a message, producing either a signature (multi-signature) or a partial signature (FROST). The reason why only a single signer is measured, instead of the time it takes for all signers to sign combined, is due to the experimental setup not being a concurrent benchmark, which results in times not applicable to real life systems, where signers will sign a message in parallel.

3.4.1.3. Aggregation Time (Threshold Signatures)

This metric is exclusive to the threshold signature scheme. Since the result of a signing function is a partial signature, these needs to be collected by a leader and then aggregated into a final signature. It is measured from the moment the leader receives the required threshold t of valid signature shares to the point where the final, aggregated signature is produced.

3.4.1.4. Verification Time

The last phase is the computation time required to cryptographically validate the final signature. For the multi-signature scheme, this involves the verifying of each of the t individual signatures withing the certificate, each using their own individual public key. For FROST, it involves the verifying of the single aggregated signature, using the joint public key.

3.4.2. Metric 2: Final Signature Size

This metric is crucial as it directly affects the message size, the bandwidth consumption and the long-term storage costs. These are important factors in large-scale distributed systems. The size of the multi-signature certificate was compared against the constant size of the single aggregated FROST signature. The size of the final certificate/signature was measured in bytes.

3.4.3. Metric 3: HotStuff QC Computation Time

Lastly, to see how multi-signature and ROAST compare in a more realistic scenario, a OMNeT++ simulation was ran, in order to determine how a real-life implementation in a BFT protocol would be affected by the use of each signature scheme.

The metric measured was the time it takes for a leader to create a QC . It is measured from the first message the leader sends to every signer, signalling the beginning of a signing session, to the point it receives the last signature and creates the QC .

3.5. Benchmark Procedure

The benchmark used one primary independent variable: The system size n. The threshold was directly derived from the system size with the following formula: $t = \frac{(2\times n+1)+2}{3}$. This formula always gave a value of t higher than $\frac{2}{3}n$, which is required as mentioned in Chapter 2.4.

The following threshold t (minimum amount of nodes required to create a signature/certificate) and system sizes n (the total number of nodes in the system) were chosen for the benchmarks, written as *t-out-of-n*, meaning t nodes from a total of n are required to produce a signature/certificate:

3-out-of-4, *7-out-of-10*, *21-out-of-30*, *34-out-of-50*, *67-out-of-100*, *334-out-of-500* and *667-out-of-1000*.

This selection gives a good range, from small local systems, to intermediate systems up until large multi-continental systems.

The controlled variables used was a fixed-length message “HELLO WORLD”, and each phase was ran for a total of 100 iterations.

The data was collected by the inbuilt CSV output that Criterion provides. Note that this feature was deprecated after Criterion v0.4.0, thus an older version that still supported it was used instead.

All tests were conducted in an optimistic scenario, meaning no network latency and no Byzantine faults. This was chosen as it provides a pure computational performance baseline to compare from.

The benchmark code is freely available on GitHub [12].

3.6. Simulation Procedure

The simulation used the same independent variable as the benchmark (the system size), but with the difference than the threshold and system sizes were limited to a maximum of *67-out-of-100* nodes. The reason being that the time required to run the simulation was too costly with the limitation of the hardware used.

A network one-way latency of 10ms was chosen, due to it being roughly the average network latency in Stockholm [13].

The data was collected in a CSV file, measured in seconds.

All tests were conducted without any Byzantine faults.

Chapter 4
Results and Analysis

4.1. Multi-signature and FROST Benchmark Results

The results in Figure 4.1 show the results of the benchmark, shown as a median value from the 100 iterations. We can clearly see that by a wide lead, the initialisation for FROST is by far the costliest phase of them all, dwarfing everything else to the point that they are not even visible on the graph. This comes as no surprise, since FROST's setup is far more complex than that of the multi-signature scheme, especially the DKG. However, since the initialisation is not done very often in a protocol, the massive cost of FROST is not a big concern, what is more important are the remaining phases.

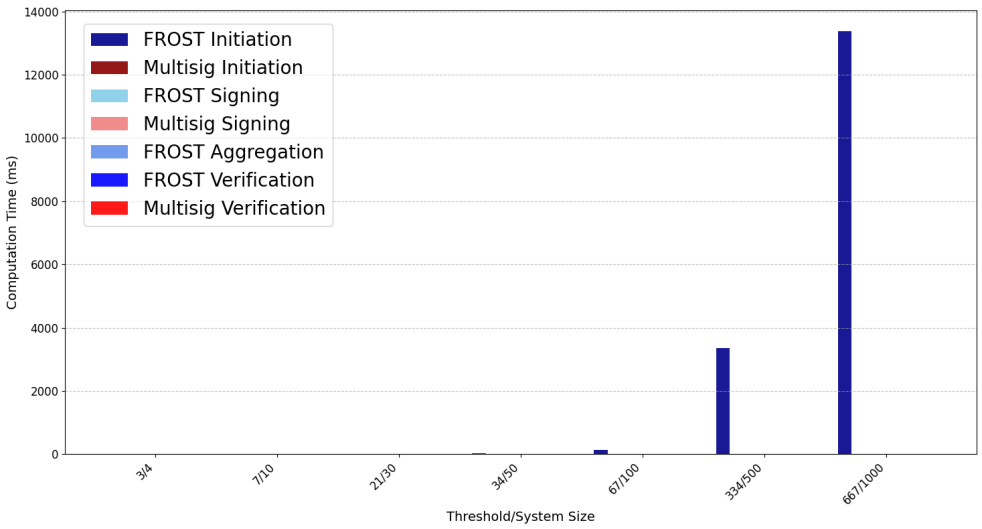

Figure 4.1: Median computation time for each phase for both schemes

In Figure 4.2, we can see two constant times. The first one is the multi-signature signing. Since every signer has their own key-pair that they use to sign, the amount of total signers in the certificate makes no difference for how long it takes to sign a message for an individual signer. From the perspective of the signer, they are the only signer in the session, thus no matter how big the system, the signing duration will stay constant. The

second constant phase is the FROST verification. Since a TSS will eventually aggregate all the partial signatures into one final singular signature, the product is indistinguishable from that which is generated by one individual signer. This means a constant size of the signature, which means a constant verification time of it as well.

The rest of the graph shows that FROST's complexity comes at a cost. The signing for FROST is longer than that of the multi-signature scheme, and it rapidly grows with the increase of the system size. FROST also introduces the additional aggregation phase, further increasing the total time for the protocol to finish a signing session. However, by far the longest phase on the graph, especially in larger systems, is the multi-signature verification phase. In system sizes of over 100, we can see that the verification time for multi-signature is almost as long as the time for FROST signing and aggregation combined, however, there is no crossover point where FROST becomes faster than multi-signature.

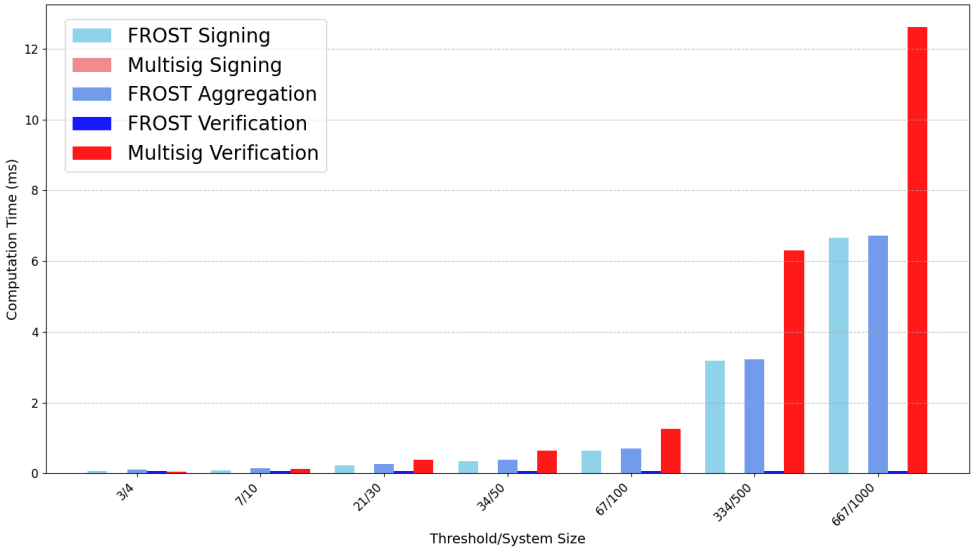

Figure 4.2: Median computation time for each phase for both schemes, excluding initialisation phase

4.2. Signature Size Results

In Figure 4.3, we can see how the signature (multi-signature certificate) size grows linearly with the amount of signers, while being constant for FROST. Since both schemes used the same underlying signature scheme (Schnorr based), each signature is 64 Bytes long, however, since in multi-signature, the

certificate is created upon reaching the threshold t, the information about which specific t out of n signers participates is unknown. So this additional information needs to be provided in the certificate, in this case the public key (32 Bytes) used for verification for each signature. This means for each additional signer, the certificate grows by 96 Bytes. In smaller systems, if the final signature is 64 or 640 Bytes might not matter, but when the system grows in the hundreds, the certificate reaches numbers in the tens of thousands of Bytes.

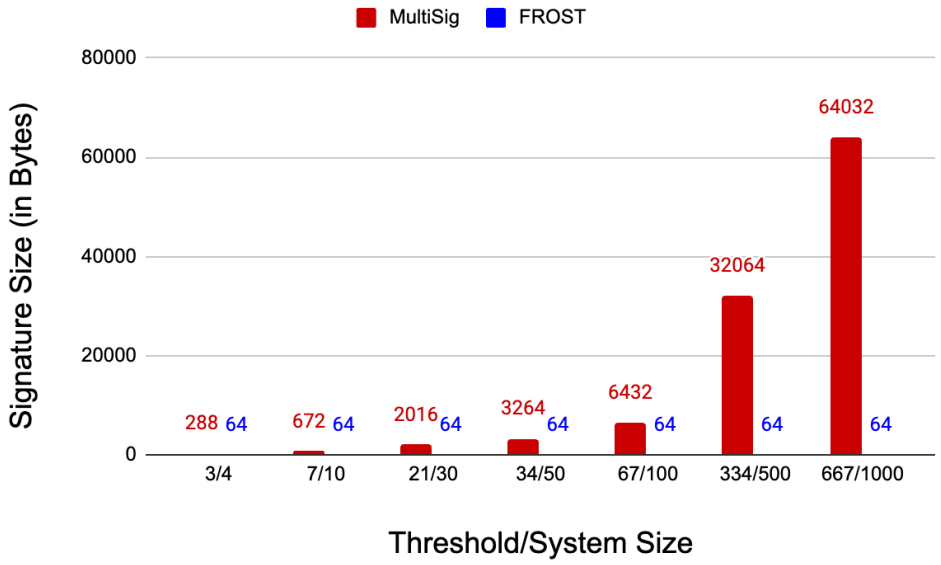

Figure 4.3: Final FROST signature/multi-signature certificate size in Bytes

4.3. OMNeT++ Simulation Results

Lastly, in Figure 4.4, we can see how each signature scheme impacts HotStuff. As expected from the previous benchmark, the clear performance winner is the multi-signature scheme. Across the different system sizes, it grows very little, unlike ROAST, which doubles from 3/4 to 67/100. However, we can see that the largest bottleneck in both schemes aren't the actual signature schemes themselves, but the network latency.

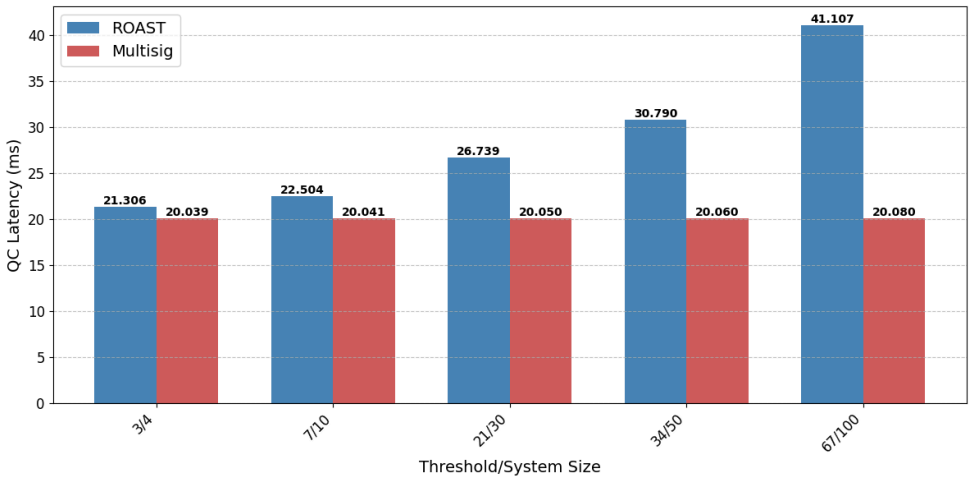

Figure 4.4: Median computation time for QC creation in HotStuff for each signature scheme

Chapter 5
Discussion

5.1. No Universal Best Signature Scheme

There is no signature scheme that is better than the other, but instead each signature scheme has its strengths and weaknesses. Which one should be used will vary depending on the system size, needs and priorities.

A system that needs fast signing or does frequent initialisations would be better off with a regular multi-signature scheme, due to the costly FROST initialisation. However, if initialisation is not a concern, and the system size is large, multi-signature might not be that much faster when it comes to signing.

If the final signature needs to be verified after creation, the long verification time for multi-signature might close the gap between the two schemes. We can see that for example, at the size of 67/100, the Multi-signature verification takes 1260µs to complete, while the combined value of FROST signing (645µs) and aggregation (695µs) adds up to 1340µs. This is very close to the multi-signature verification time (see Appendix A for all specific measurements). However, while the gap decreases with system size, in the values measured, there is no crossover point where FROST becomes faster than the multi-signature scheme.

5.2. Threshold Signatures: Compact and Efficient Verification

The biggest advantage threshold schemes have over multi-signature schemes is arguably the verification and signature size. With the growth of the system, a multi-signature certificate will become larger with the increasing number of signers.

Not only does this increase the verification time, since every single signature in the certificate has to be individually verified, but the size can cause problems in systems with limited memory. On top of this, it also affects network

transmission, with larger signatures leading to larger message payloads that take longer to send.

In systems that might store signatures for a long time and might need to verify them frequently, threshold signatures seem like a better choice.

5.3. Flexibility vs. Predefined Signers

A problem with threshold signatures is the need for the set of signers participating in the creation of a signature to be predefined before the signing. This requirement makes it far less flexible than multi-signatures, which are robust in the way that once the threshold t is met, the signature can be created.

This simple but effective way of ensuring the protocol is robust against Byzantine faults is what made it the go-to scheme to use in distributed systems that need robust fault-tolerance. However, with threshold schemes like ROAST providing robustness, they become potentially attractive for future implementations.

5.4. Impact of Network Latency on Signature Schemes

In the HotStuff simulation, we can see that when we take into account network latency, the gap between each signature scheme is not as significant as in the benchmark. Obviously, multi-signature still wins by pure computation time (excluding the verification).

However, with the chosen latency of 10ms, we can see that even at a system size of 100, ROAST is only around two times slower than multi-signature. The network latency chosen was meant to simulate how a distributed system localised entirely in one city would perform. But in systems that might span a whole country, continent, or the whole world, the network latency would substantially increase.

When we get into the triple digits, if the creation of a QC is 500 or 520ms, it might not play a big role. In this case, the advantages of a TSS like ROAST become very apparent: for a small latency fee, you get significantly smaller signatures and faster verification time.

Chapter 6
Conclusions and Future Work

As mentioned in chapter 5, each signature scheme offers both strengths and weaknesses. This study has shown that the newer TSS offers some notable advantages over traditional multi-signature schemes, but also comes with its own costs. In the end, which one should be chosen will depend on the software architect of the system; their needs, priorities and most importantly: their allocated time. Multi-signatures are relatively simple to implement, but threshold signatures on the other hand require a bit more effort to get properly working, especially with ROAST, that has inbuilt retry mechanisms that you need to account for when designing your system. While multi-signature is faster in every scenario tested, excluding the verification, the performance advantages quickly diminishes as the system size increases, and signature size and verification time quickly become more important.

An area of improvement and suggestion for future work would be to extend this research to include scenarios with Byzantine nodes to quantitatively compare the robustness and performance recovery of ROAST against traditional multi-signature schemes. This would allow for a quantitative comparison of the recovery overhead. Byzantine faults were left out from the benchmarking and simulations mainly due to constraints of the research. With limited time and resources, the focus was placed on the performance of each signature scheme in optimistic scenarios, where Byzantine faults do not occur. While Byzantine faults are critical to real-world system design, their infrequent nature in a well-managed system means their impact on overall long-term throughput is statistically minor compared to the constant overhead of cryptographic operations and network latency.

Another suggestion of future work would be to expand on the number of protocols used. Especially in the multi-signature department. The reason why a naive multi-signature scheme was chosen is due to being the most common scheme that offers great performance without being too complex. However, variants of multi-signature, such as MuSig2 [8] or aggregated multi-signatures [14] exist, which although add complexity, come with their

own advantages. Exploring these advantages they provide, and see how they compare to the naive multi-signature and threshold signature scheme would provide more material to base the comparative analysis from.

References

[1] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. USA: Addison-Wesley Publishing Company, 2011, pp. 491–500.

[2] L. Lamport, R. Shostak, and M. Pease, 'The Byzantine Generals Problem', *ACM Transactions on Programming Languages and Systems*, pp. 382–401, Jul. 1982, [Online]. Available: https://www.microsoft.com/en-us/research/publication/byzantine-generals-problem/

[3] D. Malkhi and K. Nayak, 'Extended Abstract: HotStuff-2: Optimal Two-Phase Responsive BFT'. [Online]. Available: https://eprint.iacr.org/2023/397

[4] C. Komlo, ' Threshold Signatures ', *IEEE Security & Privacy*, vol. 22, no. 6, pp. 85–88, Nov. 2024, doi: 10.1109/MSEC.2024.3463915.

[5] C. P. Schnorr, 'Efficient signature generation by smart cards', *J. Cryptol.*, vol. 4, no. 3, pp. 161–174, Jan. 1991, doi: 10.1007/BF00196725.

[6] C. Komlo and I. Goldberg, 'FROST: Flexible Round-Optimized Schnorr Threshold Signatures', in *Selected Areas in Cryptography*, O. Dunkelman, M. J. Jacobson Jr., and C. O'Flynn, Eds., Cham: Springer International Publishing, 2021, pp. 34–65.

[7] T. Ruffing, V. Ronge, E. Jin, J. Schneider-Bensch, and D. Schröder, 'ROAST: Robust Asynchronous Schnorr Threshold Signatures', in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '22. Los Angeles, CA, USA: Association for Computing Machinery, 2022, pp. 2551–2564. doi: 10.1145/3548606.3560583.

[8] J. Nick, T. Ruffing, and Y. Seurin, 'MuSig2: Simple Two-Round Schnorr Multi-Signatures'. [Online]. Available: https://eprint.iacr.org/2020/1261

[9] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, 'HotStuff: BFT Consensus with Linearity and Responsiveness', in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, in PODC '19. Toronto ON, Canada: Association for Computing Machinery, 2019, pp. 347–356. doi: 10.1145/3293611.3331591.

[10] S. Susnjara and I. Smalley, 'What is blockchain?', *IBM*, 2025, Accessed: May 02, 2025. [Online]. Available: https://www.ibm.com/think/topics/blockchain

[11] nickfarrow, 'ROAST: Robust Asychronous Schnorr Threshold Signatures'. [Online]. Available: https://github.com/nickfarrow/roast

[12] Kozarsson, 'bscThesis: a BFT signature scheme benchmarking repository'. [Online]. Available: https://github.com/Kozarsson/bscThesis

[13] SpeedGeo, 'Internet Speed in Stockholm'. Accessed: Aug. 18, 2025. [Online]. Available: https://www.speedgeo.net/statistics/sweden/stockholm

[14] D. Boneh, C. Gentry, B. Lynn, and H. Shacham, 'Aggregate and Verifiably Encrypted Signatures from Bilinear Maps', in *Advances in Cryptology — EUROCRYPT 2003*, E. Biham, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 416–432.

Appendix A
Computation Time

Table 1.1: Median computation time for Multi-signature

Threshold/System Size	Initialisation	Signing	Verification
3/4	33.9µs	8.86µs	56.5µs
7/10	84.6µs	8.86µs	132µs
21/30	255µs	8.90µs	395µs
34/50	423µs	8.88µs	641µs
67/100	847µs	8.88µs	1260µs
334/500	4240µs	8.88µs	6310µs
667/1000	8480µs	8.88µs	12600µs

Table 1.2: Median computation time for FROST

Threshold/ System Size	Initialisation	Signing	Aggregation	Verification
3/4	478µs	60.4µs	111µs	64.2µs
7/10	1950µs	95.9µs	147µs	64.2µs
21/30	14200µs	222µs	271µs	64.2µs
34/50	36600µs	345µs	396µs	64.2µs
67/100	139ms	645µs	695µs	64.2µs
334/500	3360ms	3180µs	3220µs	64.2µs
667/1000	13300ms	6.65ms	6.71ms	64.1µs

TRITA – CBH-GRU-EX 2025:318
Stockholm, Sweden 2026
www.kth.se